

TALLER DOM

YOJANES CHOCUE VELASCO

SEDE

SENA CTPI

Tercer Trimestre

Popayán/Cauca

09/10/2026

TALLER DOM

YOJANES CHOCUE VELASCO

SEDE

SENA CTPI

Instructor:

Alexandra Muños

Materia

Arquitectura

Tercer Trimestre

Popayán/Cauca

09/10/2026

1. ¿Qué significa DOM y qué palabras componen esta sigla?

Significa **Document Object Model**.

- **Document (Documento):** Representa el archivo HTML o XML cargado en el navegador.
- **Object (Objeto):** Cada elemento, atributo o fragmento de texto dentro del documento se convierte en un objeto manipulable en JavaScript.
- **Model (Modelo):** Define la estructura jerárquica en forma de árbol con la que se organizan estos objetos.

2. ¿Qué es el DOM y para qué sirve en el desarrollo web?

Es una interfaz de programación (API) para documentos HTML y XML. Sirve como puente entre el lenguaje de programación (JavaScript) y la estructura visible de la página web, permitiendo acceder, modificar, crear o eliminar elementos, contenidos, estilos y reaccionar a interacciones en tiempo real sin recargar la página.

3. ¿Cómo representa el navegador un documento HTML mediante el DOM?

El navegador lee (parsea) el código fuente HTML y construye una representación lógica en memoria con estructura jerárquica de árbol (Árbol DOM). En este árbol, la raíz es `document` y cada etiqueta, atributo o texto se convierte en un nodo interconectado.

4. ¿Qué relación existe entre HTML, CSS, JavaScript y el DOM?

- **HTML:** Proporciona la estructura inicial y estática.
- **CSS:** Define el diseño visual y los estilos aplicables.
- **DOM:** Traduce la estructura HTML en objetos navegables por código.
- **JavaScript:** Utiliza la API del DOM para manipular la estructura (HTML) y la apariencia (CSS) dinámicamente.

5. ¿Qué es un nodo en el DOM? Explique la diferencia entre un nodo de elemento, un nodo de texto y un nodo de documento

.

Un **nodo** es el punto básico de construcción dentro del árbol DOM.

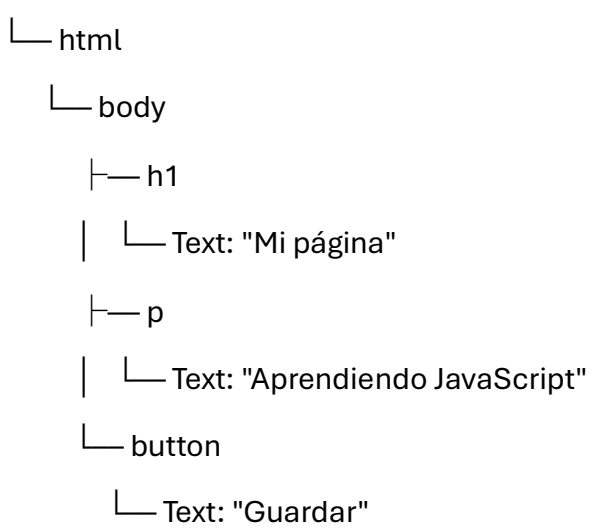
- **Nodo de Documento (`document`):** Es el nodo raíz del que se derivan todos los demás nodos.
- **Nodo de Elemento (`Element`):** Corresponde a las etiquetas HTML (ej. `<body>`, `<h1>`, `<p>`, `<button>`). Puede tener atributos y contener otros nodos.
- **Nodo de Texto (`Text`):** Representa el contenido escrito dentro de las etiquetas HTML. Es un nodo hoja (no tiene hijos).

6. ¿Qué es el árbol DOM? Dibuje un árbol que represente la estructura HTML dada

:

El **árbol DOM** es la representación gráfica/estructural de las relaciones padre-hijo-hermano de todos los nodos del documento.

Representación jerárquica de la estructura proporcionada:



7. ¿Qué ocurriría con una página web si JavaScript no pudiera acceder al DOM?

La página sería completamente estática. No se podrían realizar validaciones de formularios en tiempo real, cambiar vistas sin recargar la página (como en Single Page Applications), crear animaciones o efectos dinámicos guiados por el usuario, ni responder a eventos como clics, desplazamientos o tecleos.

8. ¿El DOM es lo mismo que el código HTML escrito por el desarrollador? Justifique su respuesta con un ejemplo

.

No. El código HTML escrito es el archivo fuente estático. El DOM es una representación viva cargada en memoria que puede ser modificada por el motor del navegador o scripts.

Ejemplo:

- **HTML original:** `<div id="contenedor"></div>`
- **JavaScript ejecutado:** `document.getElementById("contenedor").innerHTML = "<p>Hola</p>";`
- **Resultado en DOM:** Contiene la etiqueta `<p>Hola</p>`, aunque no esté en el código HTML original guardado en el archivo `.html`.

Parte 2. Selección y manipulación de elementos

Métodos y Propiedades

- **`getElementById()`**
  - *Investigación:* Selecciona un único elemento buscando coincidencia directa con su atributo `id`.
  - *Devuelve/Modifica:* Devuelve el objeto del elemento o `null` si no existe.
  - *Ejemplo:*

```
const titulo = document.getElementById("miTitulo");
```

- *Investigación:* Selecciona todos los elementos que compartan una clase CSS.
- *Devuelve/Modifica:* Devuelve una colección HTML viva (`HTMLCollection`). Se diferencia de seleccionar un único elemento en que se debe acceder por índice (ej. `[0]`) o recorrer con un bucle.
- *Ejemplo:*

```
const items = document.getElementsByClassName("item");
items[0].style.color = "red";
```

- *Investigación:* Selecciona elementos por el nombre de su etiqueta HTML (ej. `p`, `div`, `button`).
- *Devuelve/Modifica:* Devuelve una `HTMLCollection` con todos los elementos encontrados.
- *Ejemplo:*

```
const parrafos = document.getElementsByTagName("p");
```

- *Investigación:* Utiliza sintaxis de selectores CSS (ej. `#id`, `.clase`, `div > p`) y selecciona el **primer** elemento que coincida.
- *Devuelve/Modifica:* Devuelve un único elemento o `null`.
- *Ejemplo:*

```
const primerBoton = document.querySelector(".btn-primary");
```

- *Investigación:* Selecciona **todos** los elementos que coincidan con el selector CSS.
- *Devuelve/Modifica:* Devuelve un `NodeList` estático (permite usar `.forEach()`), a diferencia de `querySelector()` que solo devuelve el primero.
- *Ejemplo:*

```
const botones = document.querySelectorAll("button.action");
botones.forEach(b => b.classList.add("activo"));
```

- *Investigación:* Permite consultar o modificar la estructura HTML interna procesando etiquetas.
- *Devuelve/Modifica:* Devuelve o reemplaza una cadena con etiquetas HTML internas.
- *Ejemplo:*
-
```
document.getElementById("caja").innerHTML = "<strong>Texto en negrita</stro
```
Investigación: Permite leer o cambiar únicamente el contenido de texto plano, ignorando o escapando etiquetas HTML.
- *Devuelve/Modifica:* Devuelve o asigna una cadena de texto puro.
- *Ejemplo:*

```
document.getElementById("caja").textContent = "Texto plano sin formato";
```

- *Investigación:* Accede a la propiedad inline de estilo del elemento para modificar CSS dinámicamente.
- *Devuelve/Modifica:* Modifica los estilos CSS directo sobre el atributo `style` del elemento.
- *Ejemplo:*

```
elemento.style.backgroundColor = "blue";
```

- *Investigación:* Proporciona métodos para manipular las clases CSS de un elemento (`add`, `remove`, `toggle`, `contains`).
- *Devuelve/Modifica:* Permite agregar (`add`), quitar (`remove`), alternar (`toggle`) o verificar (`contains`) clases.
- *Ejemplo:*

```
elemento.classList.add("activo");
elemento.classList.toggle("oculto");
```

- *Investigación:* Crea un nuevo nodo de elemento HTML en memoria.
- *Devuelve/Modifica:* Devuelve el elemento recién creado listo para insertarse en el DOM.
- *Ejemplo:*

```
const nuevoLi = document.createElement("li");
nuevoLi.textContent = "Nuevo Elemento";
```

- *Investigación:* Agrega un nodo como el último hijo de un elemento padre especificado.
- *Devuelve/Modifica:* Inserta el nodo dentro del DOM.
- *Ejemplo:*

```
document.getElementById("lista").appendChild(nuevoLi);
```

- *Investigación:* Elimina directamente el elemento del árbol DOM en el que se encuentra.
- *Devuelve/Modifica:* Remueve el nodo actual.
- *Ejemplo:*

```
const elementoAEliminar = document.getElementById("alerta");
elementoAEliminar.remove();
```

- `innerHTML` parsea la cadena asignada como marcado HTML, convirtiendo el texto con etiquetas en nodos del DOM. Presenta riesgos de seguridad de tipo XSS si se usa con entradas de usuario sin validar.
- `textContent` trata toda entrada estrictamente como texto plano. Si se incluyen etiquetas HTML, se renderizan literalmente como texto en pantalla. Es más seguro y rápido.

Ejemplo demostrativo:

```html
<div id="demo1"></div>
<div id="demo2"></div>

<script>
  const codigo = "<em>Texto con formato</em>";

  document.getElementById("demo1").innerHTML = codigo;
  // Resultado en pantalla: Texto con formato (en cursiva)

  document.getElementById("demo2").textContent = codigo;
  // Resultado en pantalla: <em>Texto con formato</em> (literalmente la etiqueta)
</script>
```

Parte 3. Investigación sobre eventos en JavaScript

3.1. Conceptos fundamentales

9. ¿Qué es un evento en JavaScript?

Un evento es una acción o suceso detectado por el navegador que ocurre debido a la interacción del usuario (clic, tecleo, movimiento del ratón) o por un cambio de estado del documento (carga completa, redimensionamiento).

10. ¿Por qué los eventos son importantes para crear páginas web interactivas?

Permiten que el sistema responda dinámicamente al comportamiento del usuario en tiempo real, ofreciendo retroalimentación visual, procesando datos y ejecutando lógica programada de forma reactiva.

11. ¿Qué relación existe entre un evento, un elemento HTML y una función de JavaScript?

- El **elemento HTML** es el objetivo sobre el cual ocurre la acción.
- El **evento** es el desencadenante o acción puntual.

- La **función de JavaScript** (callback) es la respuesta ejecutada cuando ocurre el evento sobre el elemento.

12. ¿Qué significa escuchar o detectar un evento?

Significa registrar un observador (listener) en un elemento para estar atento al momento en que dicho suceso ocurra y ejecutar el código asociado inmediatamente.

13. ¿Qué es un manejador de eventos (event handler)?

Es la función encargada de ejecutar las instrucciones cuando se activa un evento determinado.

14. ¿Qué diferencia existe entre un evento y una función?

El evento es el **suceso** que tiene lugar (ej. el clic), mientras que la función es el conjunto de **instrucciones de código** que se ejecutan como consecuencia.

15. ¿Qué es `addEventListener()` y cuál es su sintaxis general?

Es el método estándar para vincular una función manejadora a un evento en un elemento determinado.

- **Sintaxis:**

```
elemento.addEventListener("nombre_evento", funcionManejadora, useCapture);
```

16. Diferencia entre `onclick` directamente en HTML y `addEventListener()` desde JavaScript:

- `onclick` en HTML mezcla maquetación con lógica de negocio y permite solo una función por evento.
- `addEventListener()` separa por completo la lógica del código HTML y permite registrar múltiples manejadores para el mismo evento sin sobrescribir los anteriores.

17. ¿Qué sucede si se registra más de un manejador para el mismo tipo de evento y elemento?

Si se usa `addEventListener()`, todos los manejadores se ejecutan secuencialmente en el orden en que fueron registrados.

18. ¿Qué es el objeto `event` y qué información puede proporcionar?

Es un objeto pasado automáticamente al manejador que contiene datos detallados sobre la interacción, tales como: la tecla presionada (`event.key`), las coordenadas del cursor (`event.clientX`), el objetivo que detonó el evento (`event.target`) o métodos para alterar el comportamiento por defecto (`event.preventDefault()`).

3.2. Tipos de eventos

- **`click`**
  - *Ocurrencia:* Al hacer clic con el botón principal del ratón sobre un elemento.
  - *Uso real:* Enviar un botón de compra en un e-commerce.
- **`dblclick`**
  - *Ocurrencia:* Al hacer doble clic rápido en un elemento.
  - *Uso real:* Abrir un archivo en un gestor de archivos web.
- **`mouseover`**
  - *Ocurrencia:* Cuando el puntero del ratón se ubica sobre el elemento o sobre sus hijos.
  - *Uso real:* Desplegar un menú emergente.
- **`mouseout`**
  - *Ocurrencia:* Cuando el cursor sale del elemento o de uno de sus elementos hijos.
  - *Uso real:* Ocultar un menú flotante.
- **`mouseenter`**
  - *Ocurrencia:* Cuando el cursor entra en el elemento (no se activa al pasar a elementos hijos).
  - *Uso real:* Resaltar una tarjeta de producto.
- **`mouseleave`**
  - *Ocurrencia:* Cuando el cursor sale por completo del elemento.
  - *Uso real:* Restaurar el estilo original de una tarjeta de producto.
- **`keydown`**
  - *Ocurrencia:* En el instante en que se presiona una tecla hacia abajo.
  - *Uso real:* Controles de movimiento en un videojuego web.
- **`keyup`**
  - *Ocurrencia:* Al soltar la tecla presionada.
  - *Uso real:* Iniciar una búsqueda automática al terminar de teclear.
- **`input`**
  - *Ocurrencia:* De forma inmediata cada vez que el valor de un `<input>`, `<textarea>` o `<select>` cambia.

    - *Uso real:* Contador de caracteres en tiempo real para una publicación.
- **`change`**
    - *Ocurrencia:* Cuando el valor de un campo ha cambiado y pierde el foco (o se selecciona una opción en un `<select>`).
    - *Uso real:* Actualizar la ciudad seleccionada según el país en un formulario.
- **`submit`**
    - *Ocurrencia:* Al presionar el botón de envío o dar Enter dentro de un formulario.
    - *Uso real:* Validar datos antes de enviar un formulario de inicio de sesión.
- **`focus`**
    - *Ocurrencia:* Cuando un elemento recibe la atención o el cursor del teclado.
    - *Uso real:* Mostrar una caja con sugerencias o un tooltip de ayuda.
- **`blur`**
    - *Ocurrencia:* Cuando un campo pierde la atención o el foco.
    - *Uso real:* Validar si el campo de correo electrónico ingresado es válido inmediatamente al salir de él.
- **`load`**
    - *Ocurrencia:* Cuando el documento completo o un recurso (como una imagen) se ha terminado de descargar.
    - *Uso real:* Ocultar la pantalla de carga (spinner) de la aplicación.

3.3. Preguntas de profundización

19. ¿Por qué un botón puede utilizar `click`, mientras que un campo de texto puede utilizar `input`?

Un botón representa una acción atómica puntualmente ejecutada al oprimirlo (`click`), mientras que un campo de texto requiere procesar progresivamente la información caracter por caracter a medida que ingresa (`input`).

20. ¿Qué diferencia existe entre `mouseenter` y `mouseover`?

`mouseover` se propaga por burbujeo y se dispara tanto al entrar al elemento como al moverse entre sus nodos hijos, mientras que `mouseenter` no burbujea y responde únicamente cuando el puntero ingresa a los límites del elemento contenedor.

21. ¿Qué diferencia existe entre keydown y keyup?

keydown ocurre en el momento exacto en que la tecla es presionada (se repite si se mantiene oprimida), mientras que keyup se dispara una única vez al soltar la tecla.

22. ¿Por qué se utiliza `preventDefault()` al trabajar con determinados formularios?

Para cancelar la recarga de la página o el envío tradicional por defecto del navegador, permitiendo validar los campos mediante JavaScript o procesar los datos asincrónicamente mediante AJAX/Fetch.

23. ¿Qué sucede cuando se ejecuta `event.target`? Construya un ejemplo.

Identifica de forma exacta el nodo/elemento específico donde se originó el evento.

- **Ejemplo:**

```
document.addEventListener("click", function(event) {
  console.log("Hiciste clic en el elemento:", event.target);
});
```

24. ¿Qué diferencia existe entre `event.target` y `event.currentTarget`?

- `event.target` es el elemento específico sobre el cual se hizo la interacción inicial.
- `event.currentTarget` es el elemento que tiene asignado el escuchador de eventos (`addEventListener`) que está ejecutando la función en ese momento.

25. ¿Qué son la propagación de eventos, el bubbling y la captura?

Son las fases de viaje de un evento en la jerarquía del DOM:

1. **Captura (Capturing phase):** El evento desciende desde la raíz (`document`) hacia el elemento destino.
2. **Burbujeo (Bubbling phase):** Tras impactar en el objetivo, el evento asciende desde el elemento destino de vuelta hacia la raíz. Por defecto, los eventos se procesan en esta fase.

26. ¿En qué consiste la delegación de eventos y cuándo podría ser útil?

Consiste en añadir un único escuchador de eventos a un elemento padre en lugar de registrarlo individualmente en cada uno de sus hijos. Es ideal cuando se tienen listas de elementos dinámicos o que se agregan y eliminan constantemente en tiempo de ejecución.

Parte 4. Taller práctico de investigación

Ejercicio 1. Botón interactivo

```
<p id="parrafoOriginal">Texto sin cambiar</p>
<button id="btnCambiar">Cambiar Texto</button>

<script>
  const parrafo = document.getElementById("parrafoOriginal");
  const boton = document.getElementById("btnCambiar");

  boton.addEventListener("click", () => {
    parrafo.textContent = "¡El texto ha sido modificado exitosamente!";
  });
</script>
```

- **Respuestas de investigación:**
  - **Selección del párrafo:** Mediante `document.getElementById("parrafoOriginal")`.
  - **Evento utilizado:** Evento `click`.
  - **Propiedad modificada:** `textContent`.

Ejercicio 2. Cambio de apariencia

```
<section id="miSeccion" style="padding: 20px; border: 1px solid #ccc;">
  Contenido de la sección
</section>
<button id="btnCambiarColor">Cambiar Color</button>
<button id="btnRestaurar">Restaurar</button>

<script>
  const seccion = document.getElementById("miSeccion");
  const btnCambiar = document.getElementById("btnCambiarColor");
  const btnRestaurar = document.getElementById("btnRestaurar");

  btnCambiar.addEventListener("click", () => {
    seccion.classList.add("fondo-activo");
  });

  btnRestaurar.addEventListener("click", () => {
    seccion.classList.remove("fondo-activo");
  });
</script>
```

- **Investigación:** Es mejor utilizar clases CSS con `classList`. Esto permite mantener la separación entre la estructura/lógica (JS) y la presentación (CSS), facilitando la reusabilidad, escalabilidad y mantenimiento del proyecto.

Ejercicio 3. Seguimiento de escritura

```
<input type="text" id="campoTexto" placeholder="Escribe algo...">
<p id="vistaPrevia"></p>

<script>
  const campoTexto = document.getElementById("campoTexto");
  const vistaPrevia = document.getElementById("vistaPrevia");

  campoTexto.addEventListener("input", (e) => {
    vistaPrevia.textContent = e.target.value;
  });
</script>
```

- **Investigación:** Es apropiado usar `input` porque reacciona instantáneamente a cualquier cambio en la entrada del texto (tecleo, pegado, eliminación), a diferencia del `click`. Con `value` obtenemos el texto almacenado en el campo.

Ejercicio 4. Lista dinámica

```
<input type="text" id="nuevoItem" placeholder="Nuevo elemento">
<button id="btnAgregar">Agregar</button>
<ul id="listaElementos"></ul>

<script>
  const entrada = document.getElementById("nuevoItem");
  const btnAgregar = document.getElementById("btnAgregar");
  const lista = document.getElementById("listaElementos");

  btnAgregar.addEventListener("click", () => {
    if (entrada.value.trim() === "") return;

    const li = document.createElement("li");
    li.textContent = entrada.value + " ";

    const btnEliminar = document.createElement("button");
    btnEliminar.textContent = "Eliminar";
    btnEliminar.addEventListener("click", () => li.remove());
```

```
    li.appendChild(btnEliminar);
    lista.appendChild(li);
    entrada.value = "";
  });
</script>
```

- **Investigación:**
  - `createElement()` crea un nodo elemento.
  - `appendChild()` lo inserta en un nodo contenedor.
  - `remove()` destruye el nodo.
  - El evento de eliminación se asigna a cada botón individual al momento de ser instanciado dinámicamente.

Ejercicio 5. Formulario con validación

```
<form id="formularioWeb">
  <input type="text" id="nombre" placeholder="Nombre"><br>
  <input type="email" id="correo" placeholder="Correo"><br>
  <input type="password" id="clave" placeholder="Contraseña"><br>
  <button type="submit">Enviar</button>
  <p id="mensajeError" style="color:red;"></p>
</form>

<script>
  const form = document.getElementById("formularioWeb");
  const errorMsg = document.getElementById("mensajeError");

  form.addEventListener("submit", (e) => {
    e.preventDefault();
    const nombre = document.getElementById("nombre").value.trim();
    const correo = document.getElementById("correo").value.trim();
    const clave = document.getElementById("clave").value.trim();
```

```
    if (!nombre || !correo || !clave) {
      errorMsg.textContent = "Todos los campos son obligatorios.";
    } else {
      errorMsg.style.color = "green";
      errorMsg.textContent = "Formulario enviado correctamente.";
    }
  });
</script>
```

- **Investigación:** `submit` detiene el flujo al enviar datos. `preventDefault()` evita el refresco automático. Las validaciones HTML son rápidas pero fáciles de eludir, mientras que la validación en JS permite reglas complejas y personalización total de errores.

Parte 5. Análisis de código

Basado en el código analizado de la página 4 del documento:

27. ¿Qué elementos se seleccionan mediante `getElementById()`?

El título <h1> con ID "titulo", el párrafo <p> con ID "mensaje" y el botón <button> con ID "boton".

28. ¿Qué función cumple addEventListener() en este código?

Registra la escucha para el evento clic en el botón y ejecuta la función de actualización del título y párrafo.

29. ¿Qué sucedería si se reemplaza click por dblclick?

La acción ya no respondería al hacer un clic simple, requiriendo un doble clic rápido sobre el botón para ejecutarse.

30. ¿Qué cambia cuando se ejecuta la función?

- El contenido del párrafo pasa a ser: "El botón fue presionado".
- El color del texto del título cambia a azul ("blue").

31. ¿Qué información muestra console.log(event.target)?

Imprime en la consola de desarrollo la referencia al elemento HTML exacto que recibió el clic: <button id="boton">Presionar</button>.

32. ¿Qué pasaría si el identificador del botón fuera incorrecto?

La variable boton guardaría null, por lo que al intentar ejecutar boton.addEventListener(...) JavaScript lanzaría un error de tipo TypeError: Cannot read properties of null.

33. ¿Qué errores podrían ocurrir si se intenta acceder a un elemento que no existe?

Lanzará un error de referencia nula TypeError al intentar leer o modificar propiedades (.textContent, .style, etc.) sobre un valor de retorno null.

34. Modifique el código para que el título cambie de color cada vez que se presione el botón, alternando entre azul y rojo:

```
boton.addEventListener("click", function(event) {
  mensaje.textContent = "El botón fue presionado";
  if (titulo.style.color === "blue") {
    titulo.style.color = "red";
  } else {
    titulo.style.color = "blue";
  }
  console.log(event.target);
});
```

35. Explique cómo comprobaría que el evento funciona sin depender únicamente de la apariencia visual:

Agregando un mensaje de registro en consola mediante `console.log("Evento click ejecutado correctamente");` dentro del cuerpo del manejador.

36. Proponga una mejora para separar el código JavaScript del documento HTML:

Crear un archivo externo `app.js`, mover allí todo el código dentro de las etiquetas `<script>` y vincularlo al documento mediante `<script src="app.js" defer></script>` justo antes del cierre del `</body>`.

Parte 6. Investigación aplicada a un caso real

Caso Real: Plataforma Educativa (Ej. Google Classroom / Moodle)

- **Nombre y propósito del sistema:**

Plataforma de gestión de aprendizaje para entregar tareas, realizar cuestionarios y revisar calificaciones.

- **Tres elementos de la interfaz:**
  - Un campo de texto de búsqueda de cursos.
  - Un botón para desplegar el panel de notificaciones.
  - Un formulario para adjuntar y entregar una tarea.
- **Eventos a utilizar:**
  - Campo de búsqueda: Evento `input`.
  - Botón de notificaciones: Evento `click`.
  - Formulario de entrega: Evento `submit`.
- **Acción que ejecuta JavaScript:**

    - Filtrar en tiempo real la lista de cursos mostrados sin recargar la página.
    - Alternar la visibilidad (mediante `classList.toggle('visible')`) de una caja flotante con las notificaciones.
    - Capturar el envío mediante `preventDefault()`, verificar la presencia del archivo y enviar los datos.
- **Explicación de la relación con el DOM:**

El usuario interactúa sobre elementos del DOM (botones o cajas de texto). Las funciones asociadas leen las entradas con `value` o manipulan la visibilidad agregando nodos dinámicamente (`appendChild()`), actualizando la pantalla en tiempo real.

- **Prototipo HTML de la funcionalidad:**